```
#include <iostream>
#include <string>
using namespace std;

// ========================================
// 1. QUEUE NODE (Flexible payload)
// ========================================
// Field mapping per question:
// Q1:  int id (use as x)
// Q2:  int id, string name, int extra (Age)
// Q3:  int id (MsgID), string name (Text), int extra (RetryCount)
// Q4:  string name, int extra (PartySize), string extraStr (Phone)
// Q5:  int id (DriverID), string extraStr (CarTier)
// Q6:  string name (IP), string extraStr (Endpoint), string extraStr2 (Timestamp)
// Q7:  int id (JobID), string name (DocName)
// Q8:  int id (PacketID), string name (SourceIP), string extraStr (DestIP)
// Q9:  int id (TokenNo), string name (CustomerName)
// Q10: int id (StudentID), string name (StudentName)
class Node {
public:
    int id;
    string name;
    int extra;          // Used for Age, PartySize, or RetryCount
    string extraStr;    // Used for Phone, Tier, Endpoint, DestIP, or Timestamp
    Node* next;

    Node(int i, string n = "", int ex = 0, string exStr = "") {
        id = i;
        name = n;
        extra = ex;
        extraStr = exStr;
        next = NULL;
    }
};

// ========================================
// 2. MASTER QUEUE CLASS (Singly Linked List)
// ========================================
class MasterQueue {
private:
    Node* front;
    Node* rear;
    int totalCount;

public:
    MasterQueue() {
        front = rear = NULL;
        totalCount = 0;
```

```
}

// Check Queue Status (All Questions)
bool isEmpty() {
    return front == NULL;
}

// Number of elements in queue (Q5: Waiting drivers count)
int getCount() {
    return totalCount;
}

// 1. Enqueue: Insert at Rear (O(1))
void enqueue(int id, string name = "", int extra = 0, string extraStr = "") {
    Node* nn = new Node(id, name, extra, extraStr);
    if (isEmpty()) {
        front = rear = nn;
    } else {
        rear->next = nn;
        rear = nn;
    }
    totalCount++;
    cout << ">> Enqueued successfully.\n";
}

// Overload for pointer insertion (Used by Q3: Dead-Letter Queue movement)
void enqueueNode(Node* nn) {
    nn->next = NULL;
    if (isEmpty()) {
        front = rear = nn;
    } else {
        rear->next = nn;
        rear = nn;
    }
    totalCount++;
}

// 2. Dequeue: Remove & Return from Front (O(1))
bool dequeue(int &outId, string &outName) {
    if (isEmpty()) {
        cout << ">> Underflow: Queue is empty!\n";
        return false;
    }
    Node* temp = front;
    outId = temp->id;
    outName = temp->name;

    front = front->next;
```

```
    if (front == NULL) {
        rear = NULL; // Reset rear when queue becomes empty
    }
    delete temp;
    totalCount--;
    return true;
}

// 3. Peek: View Front without removing (O(1))
void peek() {
    if (isEmpty()) {
        cout << ">> Queue is empty!\n";
        return;
    }
    cout << ">> Front element -> ID: " << front->id
         << " | Name/Info: " << front->name
         << " | Extra: " << front->extra
         << " | Details: " << front->extraStr << "\n";
}

// 4. Display: Show all active elements with position (All Questions)
void display() {
    if (isEmpty()) {
        cout << ">> Queue is empty!\n";
        return;
    }
    Node* curr = front;
    int pos = 1;
    cout << "\n--- Active Queue List (Total: " << totalCount << ") ---\n";
    while (curr != NULL) {
        cout << "Position " << pos++
             << " -> ID: " << curr->id
             << " | Name: " << curr->name
             << " | Val: " << curr->extra
             << " | Extra: " << curr->extraStr << "\n";
        curr = curr->next;
    }
    cout << "----------------------------------------\n";
}

// 5. Search and Remove by ID (Q5: Driver offline, Q6: Request cancellation)
bool removeById(int targetId) {
    if (isEmpty()) {
        cout << ">> Queue is empty!\n";
        return false;
    }

    // Target is at front
```

```
    if (front->id == targetId) {
        Node* temp = front;
        front = front->next;
        if (front == NULL) rear = NULL;
        delete temp;
        totalCount--;
        cout << ">> Record with ID " << targetId << " removed.\n";
        return true;
    }

    // Search middle or rear
    Node* prev = front;
    Node* curr = front->next;
    while (curr != NULL && curr->id != targetId) {
        prev = curr;
        curr = curr->next;
    }

    if (curr == NULL) {
        cout << ">> ID " << targetId << " not found!\n";
        return false;
    }

    prev->next = curr->next;
    if (curr == rear) {
        rear = prev; // Update rear if last element removed
    }
    delete curr;
    totalCount--;
    cout << ">> Record with ID " << targetId << " removed.\n";
    return true;
}

// 6. Search and Remove by Name / IP (Q4: Party cancellation, Q6: Blacklisted IP)
bool removeByName(string targetName) {
    if (isEmpty()) {
        cout << ">> Queue is empty!\n";
        return false;
    }

    // Target is at front
    if (front->name == targetName) {
        Node* temp = front;
        front = front->next;
        if (front == NULL) rear = NULL;
        delete temp;
        totalCount--;
        cout << ">> Entry [" << targetName << "] removed.\n";
```

```
            return true;
        }

        // Search middle or rear
        Node* prev = front;
        Node* curr = front->next;
        while (curr != NULL && curr->name != targetName) {
            prev = curr;
            curr = curr->next;
        }

        if (curr == NULL) {
            cout << ">> Entry [" << targetName << "] not found!\n";
            return false;
        }

        prev->next = curr->next;
        if (curr == rear) {
            rear = prev;
        }
        delete curr;
        totalCount--;
        cout << ">> Entry [" << targetName << "] removed.\n";
        return true;
    }

    // 7. Process Message with Retry/Dead-Letter Queue (Q3: Cloud Queue)
    void processMessageWithRetry(MasterQueue &deadLetterQueue, int maxRetries, bool
simulateSuccess) {
        if (isEmpty()) {
            cout << ">> No messages to process.\n";
            return;
        }

        Node* curr = front;
        front = front->next;
        if (front == NULL) rear = NULL;
        totalCount--;

        if (simulateSuccess) {
            cout << ">> Message ID " << curr->id << " processed successfully.\n";
            delete curr;
        } else {
            curr->extra++; // Increment retry count
            cout << ">> Processing failed for Message ID " << curr->id
                 << " (Attempt " << curr->extra << ").\n";
            if (curr->extra > maxRetries) {
                cout << ">> Exceeded max retries (" << maxRetries
```

```
                << "). Moving to Dead-Letter Queue.\n";
            deadLetterQueue.enqueueNode(curr);
        } else {
            cout << ">> Re-enqueuing at rear for retry.\n";
            enqueueNode(curr);
        }
    }
  }
};

// ========================================
// 3. MENU-DRIVEN INTERACTION
// ========================================
int main() {
   MasterQueue q;
   MasterQueue deadLetterQ; // Specifically used for Q3
   int choice;

   do {
      cout << "\n================= QUEUE MANAGEMENT SYSTEM
=================\n";
      cout << "1. Enqueue (Add to Rear)\n";
      cout << "2. Dequeue (Process/Remove from Front)\n";
      cout << "3. Peek (View Next Element)\n";
      cout << "4. Display All Active Elements with Positions\n";
      cout << "5. Check Queue Status (Is Empty? & Total Count)\n";
      cout << "6. Search & Remove by ID (Driver offline / Request cancel)\n";
      cout << "7. Search & Remove by Name/IP (Party cancel / Blacklist IP)\n";
      cout << "8. Cloud Worker: Process Front Message with Retries (Q3)\n";
      cout << "9. Display Dead-Letter Queue (Q3)\n";
      cout << "0. Exit\n";
      cout << "Enter your choice: ";
      cin >> choice;

      switch (choice) {
         case 1: {
            int id, extra = 0;
            string name = "", extraStr = "";
            cout << "Enter ID / Key (or 0 if not applicable): ";
            cin >> id;
            cout << "Enter Name / String Data (or '-' if empty): ";
            cin >> name;
            cout << "Enter Numeric Data (Age / PartySize / RetryCount, or 0): ";
            cin >> extra;
            cout << "Enter Extra Info (Phone / Tier / DestIP / Endpoint, or '-'): ";
            cin >> extraStr;

            q.enqueue(id, name, extra, extraStr);
```

```
            break;
        }

        case 2: {
            int outId;
            string outName;
            if (q.dequeue(outId, outName)) {
                cout << ">> Dequeued element -> ID: " << outId << " | Name: " << outName <<
"\n";
            }
            break;
        }

        case 3:
            q.peek();
            break;

        case 4:
            q.display();
            break;

        case 5:
            if (q.isEmpty()) {
                cout << ">> Status: Queue is EMPTY.\n";
            } else {
                cout << ">> Status: Queue has " << q.getCount() << " active element(s).\n";
            }
            break;

        case 6: {
            int targetId;
            cout << "Enter ID to search and remove: ";
            cin >> targetId;
            q.removeById(targetId);
            break;
        }

        case 7: {
            string targetName;
            cout << "Enter Name / IP to search and remove: ";
            cin >> targetName;
            q.removeByName(targetName);
            break;
        }

        case 8: {
            int maxRetries = 3;
            int simulatedStatus;
```

```
            cout << "Simulate processing for front message:\n";
            cout << "  1 = Successful execution\n";
            cout << "  0 = Failed execution\n";
            cout << "Enter result (1 or 0): ";
            cin >> simulatedStatus;
            q.processMessageWithRetry(deadLetterQ, maxRetries, simulatedStatus == 1);
            break;
        }

        case 9:
            cout << "\n--- Dead-Letter Queue (Failed Messages) ---";
            deadLetterQ.display();
            break;

        case 0:
            cout << "Exiting system.\n";
            break;

        default:
            cout << "Invalid choice! Please select again.\n";
      }
   } while (choice != 0);

   return 0;
}
```

---

---

TREEEE

```
#include <iostream>
using namespace std;

// ========================================
// 1. TREE NODE
// Flexible for: Book ID, Roll No, Patient Token,
// Employee ID, Bid, Price, Permit, Seat No, Ticket, Parcel
// ========================================
class TreeNode {
public:
   int val;
   TreeNode* left;
   TreeNode* right;

   // For Questions 5 & 5B (Threaded BST):
   // true = link is a thread, false = link is a real child
   bool lthread;
   bool rthread;
```

```
    TreeNode(int v) {
        val = v;
        left = right = NULL;
        lthread = rthread = true; // default threads for TBST
    }
};

// ========================================
// 2. LINKED LIST STACK (NO STL)
// Needed for Non-Recursive Traversals (Q2, Q4)
// ========================================
class StackNode {
public:
    TreeNode* tnode;
    StackNode* next;
    StackNode(TreeNode* n) {
        tnode = n;
        next = NULL;
    }
};

class MyStack {
private:
    StackNode* top;
public:
    MyStack() { top = NULL; }

    bool isEmpty() { return top == NULL; }

    void push(TreeNode* n) {
        StackNode* sn = new StackNode(n);
        sn->next = top;
        top = sn;
    }

    TreeNode* pop() {
        if (isEmpty()) return NULL;
        StackNode* temp = top;
        TreeNode* res = temp->tnode;
        top = top->next;
        delete temp;
        return res;
    }
};

// ========================================
// 3. LINKED LIST QUEUE (NO STL)
// Needed for Level-by-Level Traversal (Q1, Q3, Q8)
```

```
// ========================================
class QueueNode {
public:
    TreeNode* tnode;
    QueueNode* next;
    QueueNode(TreeNode* n) {
        tnode = n;
        next = NULL;
    }
};

class MyQueue {
private:
    QueueNode* front;
    QueueNode* rear;
public:
    MyQueue() { front = rear = NULL; }

    bool isEmpty() { return front == NULL; }

    void enqueue(TreeNode* n) {
        QueueNode* qn = new QueueNode(n);
        if (isEmpty()) {
            front = rear = qn;
        } else {
            rear->next = qn;
            rear = qn;
        }
    }

    TreeNode* dequeue() {
        if (isEmpty()) return NULL;
        QueueNode* temp = front;
        TreeNode* res = temp->tnode;
        front = front->next;
        if (front == NULL) rear = NULL;
        delete temp;
        return res;
    }
};

// ========================================
// 4. MASTER TREE CLASS
// ========================================
class MasterTree {
public:
    TreeNode* root;
```

```
MasterTree() {
    root = NULL;
}

// ------------------------------------------------
// [A] STANDARD BST OPERATIONS (Q1, 2, 3, 4, 6, 7, 8, 9)
// ------------------------------------------------

// 1. Insert into standard BST
TreeNode* insertBST(TreeNode* node, int v) {
    if (node == NULL) return new TreeNode(v);
    if (v < node->val)
        node->left = insertBST(node->left, v);
    else if (v > node->val)
        node->right = insertBST(node->right, v);
    return node;
}

// Helper: Find Minimum Node (Inorder Successor)
TreeNode* findMin(TreeNode* node) {
    while (node != NULL && node->left != NULL)
        node = node->left;
    return node;
}

// 2. Delete from standard BST (handles 0, 1, and 2 children)
TreeNode* deleteBST(TreeNode* node, int v) {
    if (node == NULL) return NULL;

    if (v < node->val) {
        node->left = deleteBST(node->left, v);
    } else if (v > node->val) {
        node->right = deleteBST(node->right, v);
    } else {
        // Case 1: No child
        if (node->left == NULL && node->right == NULL) {
            delete node;
            return NULL;
        }
        // Case 2: One child
        else if (node->left == NULL) {
            TreeNode* temp = node->right;
            delete node;
            return temp;
        } else if (node->right == NULL) {
            TreeNode* temp = node->left;
            delete node;
            return temp;
```

```
        }
        // Case 3: Two children (uses Inorder Successor)
        else {
            TreeNode* succ = findMin(node->right);
            node->val = succ->val;
            node->right = deleteBST(node->right, succ->val);
        }
    }
    return node;
}

// 3. Recursive Inorder (Ascending Order)
void inorder(TreeNode* node) {
    if (node == NULL) return;
    inorder(node->left);
    cout << node->val << " ";
    inorder(node->right);
}

// 4. Recursive Preorder
void preorder(TreeNode* node) {
    if (node == NULL) return;
    cout << node->val << " ";
    preorder(node->left);
    preorder(node->right);
}

// 5. Recursive Postorder
void postorder(TreeNode* node) {
    if (node == NULL) return;
    postorder(node->left);
    postorder(node->right);
    cout << node->val << " ";
}

// 6. Level by Level Traversal (Q1, Q3, Q8)
void printLevelOrder() {
    if (root == NULL) { cout << "Tree is empty!\n"; return; }
    MyQueue q;
    q.enqueue(root);
    cout << "Level-by-Level: ";
    while (!q.isEmpty()) {
        TreeNode* curr = q.dequeue();
        cout << curr->val << " ";
        if (curr->left != NULL) q.enqueue(curr->left);
        if (curr->right != NULL) q.enqueue(curr->right);
    }
    cout << "\n";
```

```
}

// 7. Non-Recursive Smallest and Largest (Q2)
void printMinMax() {
    if (root == NULL) { cout << "Tree is empty!\n"; return; }
    TreeNode* curr = root;
    while (curr->left != NULL) curr = curr->left;
    cout << "Smallest: " << curr->val << "\n";

    curr = root;
    while (curr->right != NULL) curr = curr->right;
    cout << "Largest: " << curr->val << "\n";
}

// 8. Non-Recursive Inorder using Stack (Q2)
void nonRecursiveInorder() {
    MyStack st;
    TreeNode* curr = root;
    cout << "Non-Recursive Inorder: ";
    while (curr != NULL || !st.isEmpty()) {
        while (curr != NULL) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.pop();
        cout << curr->val << " ";
        curr = curr->right;
    }
    cout << "\n";
}

// 9. Non-Recursive Preorder using Stack (Q4)
void nonRecursivePreorder() {
    if (root == NULL) return;
    MyStack st;
    st.push(root);
    cout << "Non-Recursive Preorder: ";
    while (!st.isEmpty()) {
        TreeNode* curr = st.pop();
        cout << curr->val << " ";
        if (curr->right != NULL) st.push(curr->right);
        if (curr->left != NULL) st.push(curr->left);
    }
    cout << "\n";
}

// 10. Count Leaves (Q3)
int countLeaves(TreeNode* node) {
```

```
    if (node == NULL) return 0;
    if (node->left == NULL && node->right == NULL) return 1;
    return countLeaves(node->left) + countLeaves(node->right);
}

// 11. Tree Height (Q4)
int getHeight(TreeNode* node) {
    if (node == NULL) return 0;
    int lh = getHeight(node->left);
    int rh = getHeight(node->right);
    return 1 + (lh > rh ? lh : rh);
}

// 12. Floor Value (Q7)
void printFloor(int key) {
    TreeNode* curr = root;
    int floorVal = -1;
    while (curr != NULL) {
        if (curr->val == key) {
            floorVal = curr->val;
            break;
        }
        if (curr->val > key) {
            curr = curr->left;
        } else {
            floorVal = curr->val;
            curr = curr->right;
        }
    }
    if (floorVal == -1) cout << "Floor: none\n";
    else cout << "Floor: " << floorVal << "\n";
}

// 13. Search with Comparison Count (Q9)
void searchWithComparisons(int key) {
    TreeNode* curr = root;
    int comps = 0;
    bool found = false;
    while (curr != NULL) {
        comps++;
        if (curr->val == key) {
            found = true;
            break;
        } else if (key < curr->val) {
            curr = curr->left;
        } else {
            curr = curr->right;
        }
```

```
    }
    if (found) cout << "Key " << key << " FOUND. Comparisons made: " << comps << "\n";
    else cout << "Key " << key << " NOT FOUND. Comparisons made: " << comps << "\n";
}

// 14. Parent and Sibling (Q9)
void printParentAndSibling(int key) {
    if (root == NULL || root->val == key) {
        cout << "Parent: none | Sibling: none\n";
        return;
    }
    TreeNode* parent = NULL;
    TreeNode* curr = root;

    while (curr != NULL && curr->val != key) {
        parent = curr;
        if (key < curr->val) curr = curr->left;
        else curr = curr->right;
    }

    if (curr == NULL) {
        cout << "Key not present in tree.\n";
        return;
    }

    cout << "Parent: " << parent->val << "\n";
    if (parent->left == curr && parent->right != NULL)
        cout << "Sibling: " << parent->right->val << "\n";
    else if (parent->right == curr && parent->left != NULL)
        cout << "Sibling: " << parent->left->val << "\n";
    else
        cout << "Sibling: none\n";
}

// ------------------------------------------------
// [B] THREADED BST OPERATIONS (Questions 5 & 5B)
// ------------------------------------------------

// Insert into Threaded BST
void insertTBST(int key) {
    TreeNode* nn = new TreeNode(key);
    if (root == NULL) {
        root = nn;
        return;
    }

    TreeNode* curr = root;
    TreeNode* parent = NULL;
```

```
    while (curr != NULL) {
        parent = curr;
        if (key < curr->val) {
            if (!curr->lthread) curr = curr->left;
            else break;
        } else {
            if (!curr->rthread) curr = curr->right;
            else break;
        }
    }

    if (key < parent->val) {
        nn->left = parent->left;
        nn->right = parent;
        parent->lthread = false;
        parent->left = nn;
    } else {
        nn->left = parent;
        nn->right = parent->right;
        parent->rthread = false;
        parent->right = nn;
    }
}

// TBST Inorder Successor using Threads
TreeNode* inorderSuccessorTBST(TreeNode* node) {
    if (node->rthread) return node->right;
    node = node->right;
    while (!node->lthread) {
        node = node->left;
    }
    return node;
}

// TBST Inorder Traversal (No recursion, No stack)
void inorderTBST() {
    if (root == NULL) return;
    TreeNode* curr = root;
    while (!curr->lthread) curr = curr->left;

    cout << "Threaded Inorder: ";
    while (curr != NULL) {
        cout << curr->val << " ";
        curr = inorderSuccessorTBST(curr);
    }
    cout << "\n";
}
```

```
    // TBST Preorder Traversal (No recursion, No stack)
    void preorderTBST() {
        if (root == NULL) return;
        TreeNode* curr = root;
        cout << "Threaded Preorder: ";
        while (curr != NULL) {
            cout << curr->val << " ";
            if (!curr->lthread) {
                curr = curr->left;
            } else if (!curr->rthread) {
                curr = curr->right;
            } else {
                while (curr != NULL && curr->rthread) {
                    curr = curr->right;
                }
                if (curr != NULL) curr = curr->right;
            }
        }
        cout << "\n";
    }

    // TBST Print Links and Count Threads (Q5 & Q5B)
    void printLinksAndCountThreads() {
        if (root == NULL) return;
        TreeNode* curr = root;
        while (!curr->lthread) curr = curr->left;

        int threadCount = 0;
        cout << "\n--- Node Links & Thread Count ---\n";
        while (curr != NULL) {
            cout << "Node " << curr->val << ": ";
            cout << "Left -> " << (curr->left ? to_string(curr->left->val) : "NULL")
                 << (curr->lthread ? " (Thread)" : " (Child)");
            if (curr->lthread) threadCount++;

            cout << " | Right -> " << (curr->right ? to_string(curr->right->val) : "NULL")
                 << (curr->rthread ? " (Thread)" : " (Child)") << "\n";
            if (curr->rthread) threadCount++;

            curr = inorderSuccessorTBST(curr);
        }
        cout << "Total Threads in Tree: " << threadCount << "\n";
    }
};

// ========================================
// 5. MAIN FUNCTION (MENU DRIVEN)
```

```
// ========================================
int main() {
    MasterTree tree;
    int choice, val, n;

    do {
        cout << "\n=========== TREE OPERATIONS MENU ==========\n";
        cout << " 1. Insert into standard BST\n";
        cout << " 2. Print Inorder (Ascending order)\n";
        cout << " 3. Print Preorder & Postorder\n";
        cout << " 4. Remove a value (0, 1, or 2 children)\n";
        cout << " 5. Level-by-Level Traversal (Queue)\n";
        cout << " 6. Min and Max non-recursively (Q2)\n";
        cout << " 7. Non-recursive Inorder (Stack) (Q2)\n";
        cout << " 8. Non-recursive Preorder (Stack) (Q4)\n";
        cout << " 9. Count Leaf Nodes & Height (Q3, Q4)\n";
        cout << "10. Find Floor (Q7)\n";
        cout << "11. Search with Comparison Count (Q9)\n";
        cout << "12. Find Parent and Sibling (Q9)\n";
        cout << "-------------------------------------------\n";
        cout << "13. Insert into Threaded BST (Q5, Q5B)\n";
        cout << "14. Threaded Inorder Traversal (Q5)\n";
        cout << "15. Threaded Preorder Traversal (Q5B)\n";
        cout << "16. Print Links & Count Threads (Q5, Q5B)\n";
        cout << " 0. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "How many values to insert? ";
                cin >> n;
                cout << "Enter " << n << " values: ";
                for (int i = 0; i < n; i++) {
                    cin >> val;
                    tree.root = tree.insertBST(tree.root, val);
                }
                break;

            case 2:
                cout << "Inorder (Ascending): ";
                tree.inorder(tree.root);
                cout << "\n";
                break;

            case 3:
                cout << "Preorder: "; tree.preorder(tree.root); cout << "\n";
                cout << "Postorder: "; tree.postorder(tree.root); cout << "\n";
```

```
            break;

        case 4:
            cout << "Enter value to remove: ";
            cin >> val;
            cout << "Before removal:\n";
            tree.printLevelOrder();
            tree.root = tree.deleteBST(tree.root, val);
            cout << "After removal:\n";
            tree.printLevelOrder();
            cout << "Ascending after removal: ";
            tree.inorder(tree.root);
            cout << "\n";
            break;

        case 5:
            tree.printLevelOrder();
            break;

        case 6:
            tree.printMinMax();
            break;

        case 7:
            tree.nonRecursiveInorder();
            break;

        case 8:
            tree.nonRecursivePreorder();
            break;

        case 9:
            cout << "Leaf Nodes: " << tree.countLeaves(tree.root) << "\n";
            cout << "Height: " << tree.getHeight(tree.root) << "\n";
            break;

        case 10:
            cout << "Enter value to find floor: ";
            cin >> val;
            tree.printFloor(val);
            break;

        case 11:
            cout << "Enter key to search: ";
            cin >> val;
            tree.searchWithComparisons(val);
            break;
```

```
            case 12:
                cout << "Enter key: ";
                cin >> val;
                tree.printParentAndSibling(val);
                break;

            case 13:
                cout << "How many values for TBST? ";
                cin >> n;
                cout << "Enter " << n << " values: ";
                for (int i = 0; i < n; i++) {
                    cin >> val;
                    tree.insertTBST(val);
                }
                break;

            case 14:
                tree.inorderTBST();
                break;

            case 15:
                tree.preorderTBST();
                break;

            case 16:
                tree.printLinksAndCountThreads();
                break;

            case 0:
                cout << "Done!\n";
                break;

            default:
                cout << "Invalid option!\n";
        }
    } while (choice != 0);

    return 0;
}
```

---
---

STACKKKK

```
#include <iostream>
#include <string>
using namespace std;

// 1. Stack Node with Min-Tracking for Q6
class StackNode {
public:
    int id;             // Can store Memory, Plate ID, Action ID, or ASCII char
    string text;        // Can store Function Name, URL, Action Text, Plate Name
    int currentMin;     // Stores the minimum value at and below this node (O(1) GetMin)
    StackNode* next;

    StackNode(int i, string s, int minVal) {
        id = i;
        text = s;
        currentMin = minVal;
        next = NULL;
    }
};

// 2. Master Stack Class
class MasterStack {
private:
    StackNode* top;
    int count;
    int maxCapacity; // For Q8: Stack overflow handling

public:
    MasterStack(int cap = 100) {
        top = NULL;
        count = 0;
        maxCapacity = cap;
    }

    bool isEmpty() {
        return top == NULL;
    }

    bool isFull() {
        return count >= maxCapacity;
    }

    // 1. Push
    void push(int id, string text) {
        if (isFull()) {
            cout << ">> Stack Overflow! Max capacity reached.\n";
            return;
```

```
    }

    // Determine current minimum for Q6
    int newMin = id;
    if (!isEmpty() && top->currentMin < id) {
        newMin = top->currentMin;
    }

    StackNode* nn = new StackNode(id, text, newMin);
    nn->next = top;
    top = nn;
    count++;
    cout << ">> Pushed: [" << text << " | Val/ID: " << id << "]\n";
}

// 2. Pop
bool pop(int &outId, string &outText) {
    if (isEmpty()) {
        cout << ">> Stack Underflow! Stack is empty.\n";
        return false;
    }
    StackNode* temp = top;
    outId = temp->id;
    outText = temp->text;

    top = top->next;
    delete temp;
    count--;
    return true;
}

// 3. Peek
void peek() {
    if (isEmpty()) {
        cout << ">> Stack is empty!\n";
        return;
    }
    cout << ">> Top Element: [" << top->text << " | Val/ID: " << top->id << "]\n";
}

// 4. Display
void display() {
    if (isEmpty()) {
        cout << ">> Stack is empty!\n";
        return;
    }
    StackNode* curr = top;
    int depth = 1;
```

```
        cout << "\n--- Current Stack (Top to Bottom) ---\n";
        while (curr != NULL) {
            cout << depth++ << ". " << curr->text << " (Val/ID: " << curr->id << ")\n";
            curr = curr->next;
        }
        cout << "-----------------------------------\n";
    }

    // 5. Get Minimum (Q6) - Runs in O(1) time
    void getMinimum() {
        if (isEmpty()) {
            cout << ">> Stack is empty! No minimum.\n";
            return;
        }
        cout << ">> Current Minimum Element: " << top->currentMin << "\n";
    }

    // 6. Search active item (Q2: Search function call)
    void search(string targetText) {
        if (isEmpty()) {
            cout << ">> Stack is empty!\n";
            return;
        }
        StackNode* curr = top;
        int pos = 1;
        bool found = false;
        while (curr != NULL) {
            if (curr->text == targetText) {
                cout << ">> Found \"" << targetText << "\" at position " << pos
                     << " from top (Memory/ID: " << curr->id << " KB)\n";
                found = true;
                break;
            }
            curr = curr->next;
            pos++;
        }
        if (!found) cout << ">> \"" << targetText << "\" not found in stack.\n";
    }
};

// 7. Palindrome Checker (Q10)
void checkPalindrome(string str) {
    MasterStack charStack;

    // Push each character onto stack
    for (int i = 0; i < (int)str.length(); i++) {
        charStack.push(str[i], string(1, str[i]));
    }
```

```
    // View top element using peek
    charStack.peek();

    // Pop characters and compare with original string from left to right
    bool isPal = true;
    for (int i = 0; i < (int)str.length(); i++) {
        int dummyId;
        string poppedChar;
        charStack.pop(dummyId, poppedChar);

        if (poppedChar[0] != str[i]) {
            isPal = false;
            break;
        }
    }

    if (isPal) cout << ">> Result: \"" << str << "\" is a PALINDROME.\n";
    else cout << ">> Result: \"" << str << "\" is NOT a PALINDROME.\n";
}

// ========================================
// Menu-Driven Interaction
// ========================================
int main() {
    MasterStack st(5); // Capacity 5 to demonstrate overflow
    int choice;

    do {
        cout << "\n======= UNIT 2: APPLICATION STACK MENU =======\n";
        cout << "1. Push (Plate / Function / URL / Undo Action)\n";
        cout << "2. Pop (Remove Top Element)\n";
        cout << "3. Peek (View Top Element)\n";
        cout << "4. Display All Elements\n";
        cout << "5. Get Minimum Element (Q6 - O(1))\n";
        cout << "6. Search Active Element (Q2 - Function Call)\n";
        cout << "7. Check Palindrome (Q10)\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1: {
                int id;
                string text;
                cout << "Enter Text/Name (URL, Plate, Function, Action): ";
                cin >> text;
                cout << "Enter ID/Value/Memory: ";
```

```
            cin >> id;
            st.push(id, text);
            break;
        }
        case 2: {
            int id;
            string text;
            if (st.pop(id, text)) {
                cout << ">> Popped: " << text << " (ID: " << id << ")\n";
            }
            break;
        }
        case 3:
            st.peek();
            break;
        case 4:
            st.display();
            break;
        case 5:
            st.getMinimum();
            break;
        case 6: {
            string name;
            cout << "Enter Name to search: ";
            cin >> name;
            st.search(name);
            break;
        }
        case 7: {
            string word;
            cout << "Enter string to check: ";
            cin >> word;
            checkPalindrome(word);
            break;
        }
        case 0:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
      }
   } while (choice != 0);

   return 0;
}
```

—----------------------------==========
==========

DLLLLL

# DLL

```
#include <iostream>
#include <string>
using namespace std;

// ---------- One coach = one node ----------
class Coach {
public:
    int number;
    string type;
    Coach* next;
    Coach* prev;

    Coach(int n, string t) {
        number = n;
        type = t;
        next = NULL;
        prev = NULL;
    }
};

// ---------- The whole train = doubly linked list ----------
class Train {
    Coach* head;
    Coach* tail;
    int count;

public:
    Train() {
        head = NULL;
        tail = NULL;
        count = 0;
    }

    // 1a. Add at front
    void addFront(int number, string type) {
        Coach* newCoach = new Coach(number, type);
        if (head == NULL) {
            head = newCoach;
            tail = newCoach;
        } else {
            newCoach->next = head;
            head->prev = newCoach;
            head = newCoach;
```

```
    }
    count++;
    cout << "Coach " << number << " added at front.\n";
}

// 1b. Add at rear
void addRear(int number, string type) {
    Coach* newCoach = new Coach(number, type);
    if (tail == NULL) {
        head = newCoach;
        tail = newCoach;
    } else {
        tail->next = newCoach;
        newCoach->prev = tail;
        tail = newCoach;
    }
    count++;
    cout << "Coach " << number << " added at rear.\n";
}

// 1c. Add at a specific position (position 1 = front)
void addAtPosition(int number, string type, int position) {
    if (position < 1 || position > count + 1) {
        cout << "Invalid position. Choose 1 to " << count + 1 << ".\n";
        return;
    }
    if (position == 1) {
        addFront(number, type);
        return;
    }
    if (position == count + 1) {
        addRear(number, type);
        return;
    }

    // middle insertion: stop at the coach just BEFORE the position
    Coach* before = head;
    for (int i = 1; i < position - 1; i++) {
        before = before->next;
    }
    Coach* after = before->next;

    Coach* newCoach = new Coach(number, type);
    newCoach->prev = before;
    newCoach->next = after;
    before->next = newCoach;
    after->prev = newCoach;
```

```
    count++;
    cout << "Coach " << number << " added at position " << position << ".\n";
}

// 2. Remove a coach by number
void removeCoach(int number) {
    if (head == NULL) {
        cout << "Train is empty.\n";
        return;
    }

    Coach* temp = head;
    while (temp != NULL && temp->number != number) {
        temp = temp->next;
    }
    if (temp == NULL) {
        cout << "Coach not found.\n";
        return;
    }

    if (temp == head && temp == tail) {        // only one coach
        head = NULL;
        tail = NULL;
    } else if (temp == head) {                 // first coach
        head = head->next;
        head->prev = NULL;
    } else if (temp == tail) {                 // last coach
        tail = tail->prev;
        tail->next = NULL;
    } else {                                   // middle coach
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }

    delete temp;
    count--;
    cout << "Coach " << number << " removed.\n";
}

// 3. Search a coach number
void search(int number) {
    Coach* temp = head;
    int position = 1;
    while (temp != NULL) {
        if (temp->number == number) {
            cout << "Found! Coach " << temp->number << " (" << temp->type
                 << ") at position " << position << " from the engine.\n";
            return;
```

```
            }
            temp = temp->next;
            position++;
        }
        cout << "Coach " << number << " not found.\n";
    }

    // 4. Display from LAST coach to ENGINE
    void displayLastToEngine() {
        if (tail == NULL) {
            cout << "Train is empty.\n";
            return;
        }
        cout << "\nLast Coach -> ... -> Engine\n";
        Coach* temp = tail;
        while (temp != NULL) {
            cout << "[ " << temp->number << " | " << temp->type << " ] -> ";
            temp = temp->prev;
        }
        cout << "ENGINE\n";
    }

    // Extra: Engine to last coach
    void displayEngineToLast() {
        if (head == NULL) {
            cout << "Train is empty.\n";
            return;
        }
        cout << "\nEngine -> ... -> Last Coach\n";
        cout << "ENGINE -> ";
        Coach* temp = head;
        while (temp != NULL) {
            cout << "[ " << temp->number << " | " << temp->type << " ] -> ";
            temp = temp->next;
        }
        cout << "END\n";
    }
};

string chooseType() {
    int typeChoice;
    cout << "Coach type: 1. AC  2. Sleeper  3. General\n";
    cout << "Enter type choice: ";
    cin >> typeChoice;
    if (typeChoice == 1) return "AC";
    if (typeChoice == 2) return "Sleeper";
    return "General";
}
```

```
int main() {
    Train train;
    int choice, number, position;
    string type;

    do {
        cout << "\n===== RAILWAY COACH MANAGEMENT =====\n";
        cout << "1. Add coach at front\n";
        cout << "2. Add coach at rear\n";
        cout << "3. Add coach at specific position\n";
        cout << "4. Remove a coach\n";
        cout << "5. Search a coach\n";
        cout << "6. Display last coach to engine\n";
        cout << "7. Display engine to last coach\n";
        cout << "0. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter coach number: ";
                cin >> number;
                type = chooseType();
                train.addFront(number, type);
                break;

            case 2:
                cout << "Enter coach number: ";
                cin >> number;
                type = chooseType();
                train.addRear(number, type);
                break;

            case 3:
                cout << "Enter coach number: ";
                cin >> number;
                type = chooseType();
                cout << "Enter position: ";
                cin >> position;
                train.addAtPosition(number, type, position);
                break;

            case 4:
                cout << "Enter coach number to remove: ";
                cin >> number;
                train.removeCoach(number);
                break;
```

```
            case 5:
                cout << "Enter coach number to search: ";
                cin >> number;
                train.search(number);
                break;

            case 6:
                train.displayLastToEngine();
                break;

            case 7:
                train.displayEngineToLast();
                break;

            case 0:
                cout << "Exiting program...\n";
                break;

            default:
                cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 0);

    return 0;
}
```

IN PRE POSTTTTTTT

```
#include <iostream>
#include <string>

using namespace std;

// 1. Character Stack
class Node {
public:
    char data;
    Node* next;
    Node(char c) {
        data = c;
        next = NULL;
    }
};

class Stack {
private:
    Node* top;

public:
    Stack() {
        top = NULL;
    }

    void push(char c) {
        Node* n = new Node(c);
        n->next = top;
        top = n;
    }

    char pop() {
        if (!top) return '\0';
        Node* t = top;
        char c = t->data;
        top = top->next;
        delete t;
        return c;
    }

    char peek() {
        return top ? top->data : '\0';
    }

    bool isEmpty() {
```

```
        return top == NULL;
    }
};

// 2. Integer Stack
class IntNode {
public:
    int data;
    IntNode* next;
    IntNode(int d) {
        data = d;
        next = NULL;
    }
};

class IntStack {
private:
    IntNode* top;

public:
    IntStack() {
        top = NULL;
    }

    void push(int x) {
        IntNode* n = new IntNode(x);
        n->next = top;
        top = n;
    }

    int pop() {
        if (!top) return 0;
        IntNode* t = top;
        int val = t->data;
        top = top->next;
        delete t;
        return val;
    }

    bool isEmpty() {
        return top == NULL;
    }
};

// 3. Operation Class
class Operation {
public:
    int prec(char c) {
```

```
        if (c == '^') return 3;
        if (c == '*' || c == '/') return 2;
        if (c == '+' || c == '-') return 1;
        return -1;
    }

    int power(int base, int exp) {
        int res = 1;
        for (int i = 0; i < exp; i++) res *= base;
        return res;
    }

    string in_to_post(string s) {
        Stack st;
        string result = "";
        for (int i = 0; i < (int)s.length(); i++) {
            char c = s[i];
            if (c == ' ') continue;

            // Direct inline alphanumeric check
            if ((c >= '0' && c <= '9') || (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
                result += c;
            } else if (c == '(') {
                st.push(c);
            } else if (c == ')') {
                while (!st.isEmpty() && st.peek() != '('
                 result += st.pop();
                st.pop();     // remove '('
            } else {
                while (!st.isEmpty() &&   ((prec(st.peek()) > prec(c)) || (prec(st.peek()) == prec(c)
&& c != '^')))
{
  result += st.pop();
                }
                st.push(c);
            }
        }
        while (!st.isEmpty()) result += st.pop();
        return result;
    }

    int evaluatePostfix(string post) {
        IntStack st;
        for (int i = 0; i < (int)post.length(); i++) {
            char c = post[i];

            // Direct inline digit check
            if (c >= '0' && c <= '9') {
```

```
            st.push(c - '0');
        } else {
            int val2 = st.pop();
            int val1 = st.pop();
            switch (c) {
                case '+': st.push(val1 + val2); break;
                case '-': st.push(val1 - val2); break;
                case '*': st.push(val1 * val2); break;
                case '/': st.push(val2 != 0 ? val1 / val2 : 0); break;
                case '^': st.push(power(val1, val2)); break;
            }
        }
    }
    return st.pop();
}

string in_to_pre(string s) {
    string rev = "";
    for (int i = (int)s.length() - 1; i >= 0; i--) {
        if (s[i] == '(') rev += ')';
        else if (s[i] == ')') rev += '(';
        else rev += s[i];
    }

    Stack st;
    string post = "";
    for (int i = 0; i < (int)rev.length(); i++) {
        char c = rev[i];
        if (c == ' ') continue;

        // Direct inline alphanumeric check
        if ((c >= '0' && c <= '9') || (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
            post += c;
        } else if (c == '(') {
            st.push(c);
        } else if (c == ')') {
            while (!st.isEmpty() && st.peek() != '(') post += st.pop();
            st.pop();
        } else {
            while (!st.isEmpty() &&
                   ((prec(st.peek()) > prec(c)) ||
                    (prec(st.peek()) == prec(c) && c == '^'))) {
                post += st.pop();
            }
            st.push(c);
        }
    }
    while (!st.isEmpty()) post += st.pop();
```

```
        string prefix = "";
        for (int i = (int)post.length() - 1; i >= 0; i--) prefix += post[i];
        return prefix;
    }
};

int main() {
    Operation op;
    string expr;

    cout << "Enter Infix Expression: ";
    cin >> expr;

    string post = op.in_to_post(expr);
    string pre = op.in_to_pre(expr);

    cout << "\nPostfix Expression: " << post << endl;
    cout << "Prefix Expression:  " << pre << endl;
    cout << "Evaluation Result:  " << op.evaluatePostfix(post) << endl;

    return 0;
}
```

```
================================================================================
================================================================================
========
```

SLLLLLLL

```
#include <iostream>
#include <string>
using namespace std;

// 1. SLL Node
class SLLNode {
public:
    int id;             // Applicant ID, Passenger ID, Book ID, Roll No, Patient ID, Song ID
    string name;        // Name, Title, Illness, etc.
    string extraStr;    // Division, Author, Artist, etc.
    int extraInt;       // Age, Entrance Score, Duration, etc.
    bool status;        // For Q7: Availability Status (true = available)
    SLLNode* next;

    SLLNode(int i, string n, string exS = "", int exI = 0, bool stat = true) {
        id = i;
        name = n;
        extraStr = exS;
        extraInt = exI;
        status = stat;
        next = NULL;
    }
};

// 2. Master SLL Class
class MasterSLL {
public:
    SLLNode* head;

    MasterSLL() {
        head = NULL;
    }

    bool isEmpty() {
        return head == NULL;
    }

    // 1. Add at Rear (End of list)
    void insertRear(int id, string name, string exS = "", int exI = 0, bool stat = true) {
        SLLNode* nn = new SLLNode(id, name, exS, exI, stat);
        if (isEmpty()) {
            head = nn;
        } else {
            SLLNode* curr = head;
            while (curr->next != NULL) {
```

```
            curr = curr->next;
        }
        curr->next = nn;
    }
    cout << ">> Record [" << name << "] added successfully.\n";
}

// 2. Add at Front (Beginning of list)
void insertFront(int id, string name, string exS = "", int exI = 0, bool stat = true) {
    SLLNode* nn = new SLLNode(id, name, exS, exI, stat);
    nn->next = head;
    head = nn;
    cout << ">> Record [" << name << "] added at front.\n";
}

// 3. Delete from Front (Q3: Ticket confirmed, Q10: Patient consulted)
void deleteFront() {
    if (isEmpty()) {
        cout << ">> List is empty! Nothing to remove.\n";
        return;
    }
    SLLNode* temp = head;
    head = head->next;
    cout << ">> Removed front record: " << temp->name << " (ID: " << temp->id << ")\n";
    delete temp;
}

// 4. Delete by ID (Q6: Song, Q7: Book, Q9: Student)
void deleteById(int targetId) {
    if (isEmpty()) {
        cout << ">> List is empty!\n";
        return;
    }

    if (head->id == targetId) {
        deleteFront();
        return;
    }

    SLLNode* prev = head;
    SLLNode* curr = head->next;
    while (curr != NULL && curr->id != targetId) {
        prev = curr;
        curr = curr->next;
    }

    if (curr == NULL) {
        cout << ">> Record with ID " << targetId << " not found!\n";
```

```
        return;
    }

    prev->next = curr->next;
    cout << ">> Deleted record: " << curr->name << " (ID: " << curr->id << ")\n";
    delete curr;
}

// 5. Search by ID (All Questions)
void searchById(int targetId) {
    if (isEmpty()) {
        cout << ">> List is empty!\n";
        return;
    }
    SLLNode* curr = head;
    int pos = 1;
    while (curr != NULL) {
        if (curr->id == targetId) {
            cout << ">> Found at position " << pos << ":\n";
            cout << "   ID: " << curr->id << " | Name: " << curr->name
                 << " | Detail: " << curr->extraStr << " | Value: " << curr->extraInt
                 << " | Status: " << (curr->status ? "Available" : "Checked out") << "\n";
            return;
        }
        curr = curr->next;
        pos++;
    }
    cout << ">> Record with ID " << targetId << " NOT found.\n";
}

// 6. Display All Records
void display(bool onlyAvailable = false) {
    if (isEmpty()) {
        cout << ">> List is empty!\n";
        return;
    }
    SLLNode* curr = head;
    int pos = 1;
    cout << "\n--- Linked List Records ---\n";
    while (curr != NULL) {
        if (!onlyAvailable || curr->status) {
            cout << pos++ << ". ID: " << curr->id
                 << " | Name: " << curr->name
                 << " | Extra Info: " << curr->extraStr
                 << " | Val/Score/Age: " << curr->extraInt
                 << " | Status: " << (curr->status ? "Active/Available" : "Inactive") << "\n";
        }
        curr = curr->next;
```

```
        }
        cout << "---------------------------\n";
    }

    // 7. Sort by ID using Bubble Sort on SLL Data (Q2: College Admission)
    void sortById() {
        if (isEmpty() || head->next == NULL) {
            cout << ">> List has 0 or 1 element, already sorted.\n";
            return;
        }

        bool swapped;
        do {
            swapped = false;
            SLLNode* curr = head;
            while (curr->next != NULL) {
                if (curr->id > curr->next->id) {
                    // Swap data fields
                    swap(curr->id, curr->next->id);
                    swap(curr->name, curr->next->name);
                    swap(curr->extraStr, curr->next->extraStr);
                    swap(curr->extraInt, curr->next->extraInt);
                    swap(curr->status, curr->next->status);
                    swapped = true;
                }
                curr = curr->next;
            }
        } while (swapped);

        cout << ">> List sorted by ID in ascending order.\n";
    }
};

// ========================================
// Interactive Menu for SLL
// ========================================
int main() {
    MasterSLL sll;
    int choice;

    do {
        cout << "\n======== SLL MASTER MENU ========\n";
        cout << "1. Insert Record at End (Rear)\n";
        cout << "2. Insert Record at Beginning (Front)\n";
        cout << "3. Delete Front Record (Ticket confirmed / Consulted)\n";
        cout << "4. Delete by ID (Cancel / Remove)\n";
        cout << "5. Search by ID\n";
        cout << "6. Display All Records\n";
```

```
cout << "7. Display Only Available Books (Q7)\n";
cout << "8. Sort Records by ID (Q2)\n";
cout << "0. Exit\n";
cout << "Enter choice: ";
cin >> choice;

switch (choice) {
    case 1:
    case 2: {
        int id, exI;
        string name, exS;
        int statInput;
        cout << "Enter ID: "; cin >> id;
        cout << "Enter Name / Title: "; cin >> name;
        cout << "Enter Extra Info (Author / Division / Illness / Artist): "; cin >> exS;
        cout << "Enter Numeric Info (Score / Age / Attendance % / Duration): "; cin >> exI;
        cout << "Is Available/Active? (1 for Yes, 0 for No): "; cin >> statInput;

        if (choice == 1) sll.insertRear(id, name, exS, exI, statInput == 1);
        else sll.insertFront(id, name, exS, exI, statInput == 1);
        break;
    }
    case 3:
        sll.deleteFront();
        break;
    case 4: {
        int id;
        cout << "Enter ID to delete: "; cin >> id;
        sll.deleteById(id);
        break;
    }
    case 5: {
        int id;
        cout << "Enter ID to search: "; cin >> id;
        sll.searchById(id);
        break;
    }
    case 6:
        sll.display(false);
        break;
    case 7:
        sll.display(true);
        break;
    case 8:
        sll.sortById();
        sll.display(false);
        break;
    case 0:
```

```
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
    }
  } while (choice != 0);

  return 0;
}
```

==========
==========
=

GRAPHHHH

```
#include <iostream>
#include <string>
using namespace std;

const int INF = 999999;

// ==================== 1. ADJACENCY LIST USING SLL ====================
class AdjNode {
public:
  int dest;
  int weight;
  AdjNode* next;

  AdjNode(int d, int w = 1) {
    dest = d;
    weight = w;
    next = NULL;
  }
};

// ==================== 2. CUSTOM QUEUE FOR BFS ====================
class QueueNode {
public:
  int data;
  QueueNode* next;
  QueueNode(int val) { data = val; next = NULL; }
};

class SimpleQueue {
private:
  QueueNode* front;
  QueueNode* rear;
```

```
public:
    SimpleQueue() { front = rear = NULL; }
    bool isEmpty() { return front == NULL; }

    void enqueue(int val) {
        QueueNode* nn = new QueueNode(val);
        if (isEmpty()) front = rear = nn;
        else {
            rear->next = nn;
            rear = nn;
        }
    }

    int dequeue() {
        if (isEmpty()) return -1;
        QueueNode* temp = front;
        int val = temp->data;
        front = front->next;
        if (front == NULL) rear = NULL;
        delete temp;
        return val;
    }
};

// ==================== 3. MASTER GRAPH CLASS ====================
class TraversalGraph {
private:
    int V;
    AdjNode** head; // Array of SLL heads

    void dfsHelper(int u, bool visited[]) {
        visited[u] = true;
        cout << u << " ";

        // Traverses neighbors in natural order of list
        AdjNode* curr = head[u];
        while (curr != NULL) {
            int v = curr->dest;
            if (!visited[v]) {
                dfsHelper(v, visited);
            }
            curr = curr->next;
        }
    }

public:
    TraversalGraph(int vertices) {
        V = vertices;
```

```
    head = new AdjNode*[V];
    for (int i = 0; i < V; i++) head[i] = NULL;
}

~TraversalGraph() {
    for (int i = 0; i < V; i++) {
        AdjNode* curr = head[i];
        while (curr != NULL) {
            AdjNode* temp = curr;
            curr = curr->next;
            delete temp;
        }
    }
    delete[] head;
}

// Insert edge sorted by destination ID so tie-break rule is explicit
void addEdge(int u, int v, int w = 1, bool bidirectional = true) {
    insertSortedNeighbor(u, v, w);
    if (bidirectional) {
        insertSortedNeighbor(v, u, w);
    }
}

void insertSortedNeighbor(int src, int dst, int w) {
    AdjNode* nn = new AdjNode(dst, w);
    if (head[src] == NULL || head[src]->dest >= dst) {
        nn->next = head[src];
        head[src] = nn;
        return;
    }
    AdjNode* curr = head[src];
    while (curr->next != NULL && curr->next->dest < dst) {
        curr = curr->next;
    }
    nn->next = curr->next;
    curr->next = nn;
}

// Q2(b) & Q6(b): Count connections and print degree
void printDegrees() {
    int maxDegree = -1;
    int maxNode = 0;
    cout << "\n--- Node Connection Counts ---\n";
    for (int i = 0; i < V; i++) {
        int count = 0;
        AdjNode* curr = head[i];
        while (curr != NULL) {
```

```
            count++;
            curr = curr->next;
        }
        cout << "Node " << i << ": " << count << " outgoing route(s)\n";
        if (count > maxDegree) {
            maxDegree = count;
            maxNode = i;
        }
    }
    cout << ">> Node with the MOST connections: " << maxNode
         << " (" << maxDegree << " connections)\n";
}

// Q4 & Q6: Recursive DFS (Rule: visits unvisited neighbor with smallest ID first)
void runDFS(int startNode) {
    bool* visited = new bool[V];
    for (int i = 0; i < V; i++) visited[i] = false;

    cout << "\n>> DFS Visit Order (Rule: smallest neighbour ID first): ";
    dfsHelper(startNode, visited);
    cout << "\n";

    // Q4(3): Check for unpowered substations / unreachable chambers
    bool allReached = true;
    cout << "Unreachable / In the dark: ";
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            cout << i << " ";
            allReached = false;
        }
    }
    if (allReached) cout << "None (Full network is connected)";
    cout << "\n";

    delete[] visited;
}

// Q3 & Q7: BFS Level-by-Level (Rumour days / Friend levels)
void runBFS_Levels(int startNode) {
    int* level = new int[V];
    for (int i = 0; i < V; i++) level[i] = -1;

    SimpleQueue q;
    q.enqueue(startNode);
    level[startNode] = 0;

    while (!q.isEmpty()) {
        int u = q.dequeue();
```

```
        AdjNode* curr = head[u];
        while (curr != NULL) {
            int v = curr->dest;
            if (level[v] == -1) {
                level[v] = level[u] + 1;
                q.enqueue(v);
            }
            curr = curr->next;
        }
    }

    // Print day by day / level by level
    int maxLevel = 0;
    for (int i = 0; i < V; i++) if (level[i] > maxLevel) maxLevel = level[i];

    cout << "\n--- Progression by Day / Level from Node " << startNode << " ---\n";
    for (int l = 1; l <= maxLevel; l++) {
        cout << "Day / Level " << l << ": ";
        for (int i = 0; i < V; i++) {
            if (level[i] == l) cout << i << " ";
        }
        cout << "\n";
    }

    cout << "Never hears / Not connected: ";
    bool allConnected = true;
    for (int i = 0; i < V; i++) {
        if (level[i] == -1) {
            cout << i << " ";
            allConnected = false;
        }
    }
    if (allConnected) cout << "None";
    cout << "\n";

    delete[] level;
}

// Q3(4): Suggest "People You May Know" (exact friends-of-friends with mutual count)
void suggestFriends(int target) {
    int* mutualCount = new int[V];
    bool* isDirect = new bool[V];
    for (int i = 0; i < V; i++) {
        mutualCount[i] = 0;
        isDirect[i] = false;
    }

    // Mark direct friends
```

```
    AdjNode* curr = head[target];
    while (curr != NULL) {
        isDirect[curr->dest] = true;
        curr = curr->next;
    }

    // Count mutual friends via direct friends
    curr = head[target];
    while (curr != NULL) {
        int friendNode = curr->dest;
        AdjNode* fof = head[friendNode];
        while (fof != NULL) {
            int cand = fof->dest;
            if (cand != target && !isDirect[cand]) {
                mutualCount[cand]++;
            }
            fof = fof->next;
        }
        curr = curr->next;
    }

    cout << "\n--- People You May Know for User " << target << " ---\n";
    for (int maxMutual = V; maxMutual >= 1; maxMutual--) {
        for (int i = 0; i < V; i++) {
            if (mutualCount[i] == maxMutual) {
                cout << "User " << i << " (" << maxMutual << " mutual friends)\n";
            }
        }
    }

    delete[] mutualCount;
    delete[] isDirect;
}

// Q2 & Q10: Dijkstra's Shortest Path
void dijkstra(int startNode) {
    int* dist = new int[V];
    bool* visited = new bool[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = false;
    }

    dist[startNode] = 0;

    for (int count = 0; count < V - 1; count++) {
        int minVal = INF, u = -1;
```

```
        for (int v = 0; v < V; v++) {
            if (!visited[v] && dist[v] < minVal) {
                minVal = dist[v];
                u = v;
            }
        }

        if (u == -1) break;
        visited[u] = true;

        AdjNode* curr = head[u];
        while (curr != NULL) {
            int v = curr->dest;
            int w = curr->weight;
            if (!visited[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
            curr = curr->next;
        }
    }

    cout << "\n--- Shortest Travel Time from Node " << startNode << " ---\n";
    for (int i = 0; i < V; i++) {
        cout << "To Node " << i << ": ";
        if (dist[i] == INF) cout << "unreachable\n";
        else cout << dist[i] << " mins\n";
    }

    delete[] dist;
    delete[] visited;
}

// Q10(2): Sort neighbors of a junction from nearest to farthest
void printNeighborsSorted(int u) {
    cout << "\nNeighbours of junction " << u << " (Nearest to Farthest):\n";

    // Count neighbours
    int n = 0;
    AdjNode* curr = head[u];
    while (curr != NULL) { n++; curr = curr->next; }

    if (n == 0) { cout << "No neighbours.\n"; return; }

    int* dests = new int[n];
    int* weights = new int[n];
    curr = head[u];
    for (int i = 0; i < n; i++) {
        dests[i] = curr->dest;
```

```
            weights[i] = curr->weight;
            curr = curr->next;
        }

        // Sort by weight
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (weights[j] > weights[j + 1]) {
                    swap(weights[j], weights[j + 1]);
                    swap(dests[j], dests[j + 1]);
                }
            }
        }

        for (int i = 0; i < n; i++) {
            cout << "  -> Junction " << dests[i] << " (" << weights[i] << " mins)\n";
        }

        delete[] dests;
        delete[] weights;
    }
};

// ==================== MAIN DEMO ====================
int main() {
    int V, E;
    cout << "Enter number of nodes and edges/connections: ";
    if (!(cin >> V >> E)) return 0;

    TraversalGraph g(V);
    cout << "Enter each connection (u v [weight/time]):\n";
    for (int i = 0; i < E; i++) {
        int u, v, w = 1;
        cin >> u >> v >> w; // Read weight (enter 1 if unweighted)
        g.addEdge(u, v, w, true);
    }

    g.printDegrees();

    int start;
    cout << "\nEnter starting node for traversals/search: ";
    cin >> start;

    g.runDFS(start);
    g.runBFS_Levels(start);
    g.suggestFriends(start);
    g.dijkstra(start);
    g.printNeighborsSorted(start);
```

```
    return 0;
}
```